

HALAH System and Workflow

The HALAH implementation provides a role-based patient consent management environment that operationalises the proposed multi-party healthcare data-sharing framework. The subsystem is organised around four principal functions: patient consent creation, guardian cryptographic approval, authorised doctor data access, and independent audit of the consent lifecycle. Rather than treating consent as a single approval event, the implementation represents consent as a lifecycle in which a consent request is created, independently authorised by designated parties, and cryptographically activated on a public blockchain. It is then continuously checked for validity, subsequently used to control access to specific electronic medical record (EMR) modules, and finally revoked by the patient.

In version 1.2, the activation and revocation of a consent are not confined to the application. Instead, they are executed as real transactions on the **Ethereum Sepolia** test network. Consequently, the essential state changes of a consent can be verified by any third party on a public block explorer. The overall workflow implemented by the system is therefore consistent with the proposed authorisation model as

*Create Consent → Guardian Approval → Multi-signature Validation → On-chain Consent Activation
→ Doctor Access → Consent Revocation → Independent Audit*

This sequence provides an end-to-end demonstration of how the conceptual consent model is translated into an operational healthcare data-sharing process. Each stage is documented below with a screenshot of the running system and, where the stage produces a blockchain transaction, with the corresponding transaction on Sepolia Etherscan.

1. HALAH System

The main HALAH subsystem presents itself as a Patient Consent Management Platform for History Access by Authorised Healthcare Providers. The subsystem provides a role-oriented environment in which different participants interact with the same consent lifecycle according to their respective authorisation responsibilities.

The implementation does not expose the underlying blockchain and cryptographic mechanisms as isolated technical components. Instead, these mechanisms are incorporated into the user workflow. Consequently, the subsystem presents the system from the perspective of the healthcare participants while the underlying implementation performs identity verification, consent-state checking, multi-party approval, cryptographic validation, on-chain anchoring and audit recording.

This design corresponds to the overall HALAH architecture, in which decentralised identifiers (DIDs) provide identity pointers, the consent object defines the

authorisation conditions, cryptographic signatures provide evidence of approval, and the blockchain-based ledger provides lifecycle traceability.

The consent token contract, **ConsentSBTv3**, was compiled in Remix and deployed to the Sepolia test network from the project test wallet through MetaMask. The contract implements a soulbound ERC-721 token. Transfers between accounts are blocked. Consequently, an activated consent can never be traded or moved, and burning follows the ERC-5484 consensual-burn interface. Each token stores the pseudonymous patient and requester addresses, a hash of the purpose, the validity window, a two-letter jurisdiction code and a revocation flag. The purpose is stored only as a keccak256 hash, and no medical content is written to the chain; the plaintext purpose and the EMR data remain in the application.

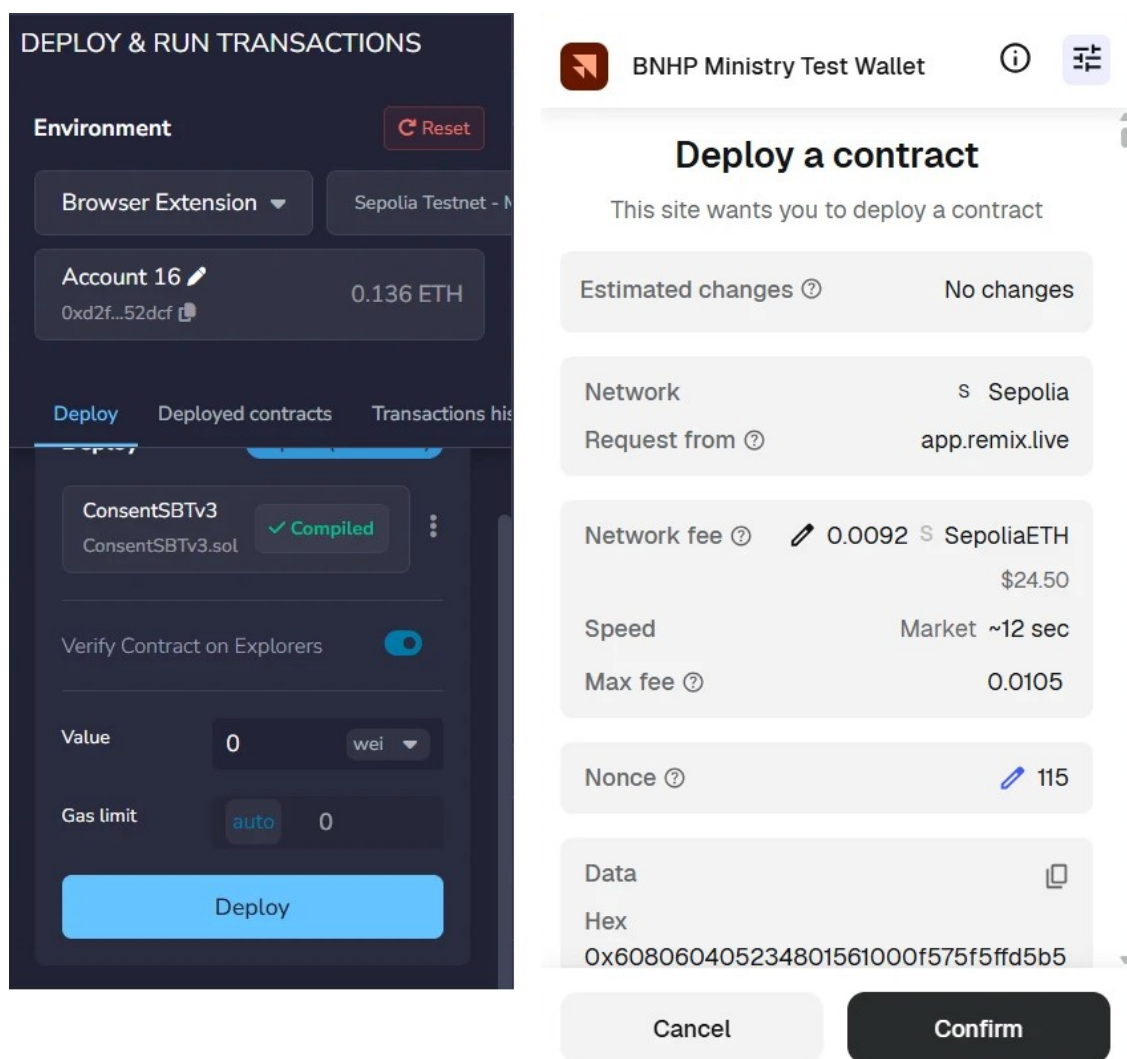


Figure 1. Deployment of ConsentSBTv3 from Remix through MetaMask on the Sepolia network.

As shown in the figure above, the deployment is an ordinary Ethereum transaction that the wallet owner must explicitly confirm: the project test wallet (displayed as BNHP Ministry Test Wallet) shows the target network (Sepolia), the estimated network

fee and the account nonce before the contract is created. This is important because it demonstrates that consent activation in HALAH ultimately rests on the same transaction-signing discipline as any other blockchain operation.

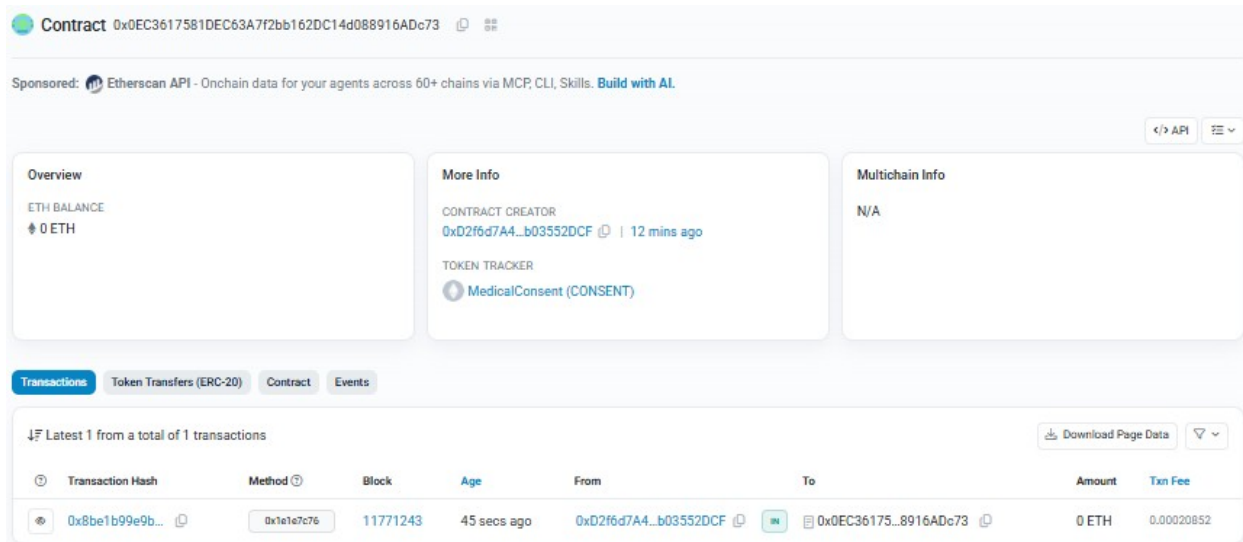


Figure 2. The deployed contract on Sepolia Etherscan, with the creator wallet, the MedicalConsent (CONSENT) token tracker and the first mint transaction.

As shown in the figure above, the deployed contract is publicly listed at [0x0EC3617581DEC63A7f2bb162DC14d088916ADc73](https://sepolia.etherscan.io/address/0x0EC3617581DEC63A7f2bb162DC14d088916ADc73) together with its creator wallet and the token tracker MedicalConsent (CONSENT). Consequently, the existence of the consent registry itself, and every activation or revocation recorded on it, can be inspected independently of the HALAH application.

The subsystem therefore serves as the entry point for demonstrating the complete authorisation process rather than functioning merely as a conventional patient-record application.

2. Create Smart Consent Contract: Patient Subsystem

The first functional subsystem is the Create Smart Consent Contract module. This module represents the patient-side initiation of a consent transaction and corresponds directly to the construction of the proposed consent object.

The subsystem requires the patient to specify the Patient DID, Authorised Doctor DID, Data Access Scope, Designated Guardian DIDs, the Jurisdiction, and the temporal validity of the consent. These fields collectively operationalise the proposed consent representation:

$$C = (I, A, P, T, L, E)$$

where the identity of the patient is represented through a DID, the authorised actor is explicitly specified, the permitted data scope is constrained to selected EMR modules, the temporal fields define when the authorisation becomes valid and when it expires,

the jurisdiction field constrains where the consent may be exercised, and the enforcement state records whether the consent is active or revoked.

Create Smart Consent Contract

Patient DID (Identity Pointer)
did:patient:5f01f823-c8e8-9e40-c911-54b7b1ad843a

Authorised Doctor DID
did:hospital:001

Jurisdiction (VL)
AU

Designated Guardian DIDs (Comma Separated)
did:parent:A, did:parent:B

Contract Temporal Validity
Start Time (YYYY-MM-DD HH:MM:SS)
2026-09-24 10:55:49
Expiry Time (YYYY-MM-DD HH:MM:SS)
2026-10-31 23:59:59

Data Access Scope (EMR Modules)
Observation x Medication x Condition x Procedure x

Initial Consent

Consent contract created and logged (off-chain).

```
[CONSENT CREATED - OFF-CHAIN]
• Consent ID: 04da25f5-b16f-4d9a-af39-a78f0a83b17f
• TxHash: OFF-CHAIN - anchoring happens after guardian 2/2 approval
• Block Number: pending
```

My Consents

04da25f5... → did:hospital:001 PENDING_SIGNATURE 0/2 sigs - token: - **Revoke**

Figure 3. The patient subsystem after consent creation: four EMR modules, two designated guardians, jurisdiction AU, and the state `PENDING_SIGNATURE`.

As shown in the figure above, the Patient DID provides the identity pointer for the data subject, while the Authorised Doctor DID limits the intended healthcare recipient. This is important because the implementation does not represent consent as a general permission to access a patient's medical information. Instead, the authorisation is bound to a specific identity and therefore supports subject-specific and actor-specific access control.

The Data Access Scope is particularly important to the proposed framework. Instead of granting access to the complete EMR, the patient selects particular modules; in the demonstrated transaction all four modules (Observation, Medication, Condition and Procedure) were selected. This implements the principle of purpose- and scope-limited consent and corresponds to the P component of the consent object. Version 1.2 additionally validates the input before the consent is stored: a consent with an empty data scope is rejected, and every guardian entry must be a well-formed DID.

The Designated Guardian DIDs establish the additional parties required to participate in the authorisation process. Their presence in the consent-creation subsystem demonstrates that multi-party consent is incorporated at the point at which the consent contract is constructed rather than being added after authorisation.

Finally, the Start Time and Expiry Time implement temporal constraints on the consent, and the Jurisdiction field records the legal territory (AU in the demonstrated transaction). Consequently, authorisation is not represented as a permanent or universal permission. Instead, the consent has a defined lifecycle and territory and can subsequently be evaluated by the Consent Engine according to both. At this point the consent exists only in the application ledger, in the state PENDING_SIGNATURE; nothing is written to the blockchain until the guardians have approved.

Two consent lifecycles appear in the demonstration and should be distinguished. The evaluated consent, f050c29a, was carried through the complete lifecycle — activation as **token #1**, granted access, refused access and final revocation — and all transaction evidence in this chapter follows it. Because a revoked consent cannot be photographed in its earlier states, the screenshots of consent creation, guardian approval and granted access were captured from a second, identical lifecycle (consent 04da25f5, anchored as **token #2** in block 11,771,581). The two lifecycles were executed with the same code and the same contract.

From a system perspective, this module therefore corresponds primarily to **Consent Object Construction and Patient Initiation**.

3. Guardian Cryptographic Approval

The second part is the Guardian Cryptographic Approval. As shown in the figure below, it operationalises the multi-party authorisation component of the proposed framework.

The screenshot displays the 'Guardian Cryptographic Approval' interface. At the top, it prompts the user to 'Enter Target Consent ID' with the value '04da25f5-b16f-4d9a-af39-a78f0a83b17f'. Below this, a message states 'Found active transaction for Patient: did:patient:5f01f823-c8e8-9e40-c911-54b7b1ad843a'. The next step is 'Select Your Guardian DID Identity', with 'did:parent:B' selected from a dropdown menu. A green button labeled 'Sign Consent via VC' is visible. Below the button, a green box confirms 'EIP-712 signature verified (signer 0x39B12b87...), Multi-sig: 2/2'. A yellow box indicates '[SBT MINTED - LOCAL LEDGER]' with details: 'Soulbound Token ID: SBT-LOCAL-04da25f5' and 'Status: 2/2 guardian approval reached - on-chain anchoring result shown below.' At the bottom, a blue box states 'Anchoring on Sepolia in the background (15-30 s). You may navigate away - the result is written to the ledger automatically. Refresh or open Auditor to see it.'

Figure 4. The guardian subsystem at the moment of the second approval: the EIP-712 signature is verified, the 2/2 threshold is reached, the SBT is minted in the local ledger and the Sepolia anchoring starts in the background.

As shown in the figure above, the guardian first enters the target Consent ID, after which the system retrieves the corresponding active transaction and displays the associated patient DID. The guardian then selects their own guardian DID identity and approves the consent with an EIP-712 typed-data signature, which binds the approval to this specific consent, this guardian and this network (chainId 11155111). The system verifies each signature by recovering the signer address and comparing it with the address associated with that guardian DID. A signature produced for another consent, another guardian or another chain is rejected, and the interface refuses a second signature from the same guardian.

The subsystem subsequently reports:

EIP-712 signature verified (signer 0x39B12b87...). Multi-sig: 2/2

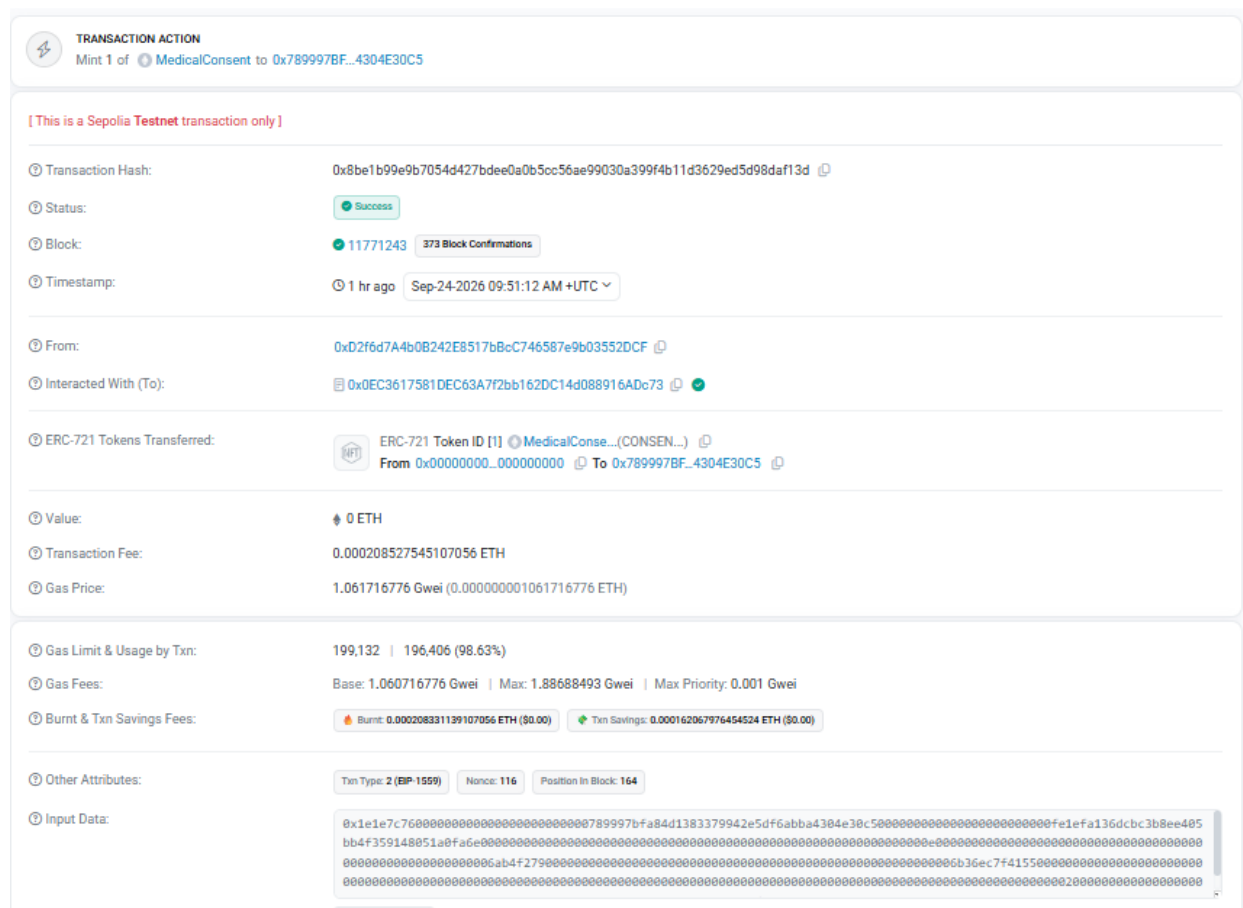
This is an important implementation-level demonstration because the consent is not activated merely because the patient created it. Instead, the system requires the predefined guardian approval condition to be satisfied. The implementation therefore enforces the logical authorisation rule:

$$ALLOW = VI \wedge VA \wedge VP \wedge VT \wedge VL$$

where identity verification, authorisation verification, purpose verification, temporal verification and jurisdictional verification must all be satisfied before access can be granted. For the demonstrated two-guardian configuration, the approval process can also be represented as:

$$\sum_{g \in G} \text{Sign}(g, C) \geq |G| = 2$$

where G represents the designated guardians and every listed guardian must provide a valid signature for activation. Once the required signatures have been obtained, the subsystem displays SBT MINTED together with the Soulbound Token identifier, and a background job submits the mint transaction to Sepolia. When the guardian later re-enters the Consent ID, the subsystem reports Already anchored on Sepolia together with the on-chain token number (token #2 for the photographed lifecycle, token #1 for the evaluated consent). This provides a visible representation of the transition from a pending consent transaction to an activated, publicly recorded consent state. The SBT therefore functions in the implementation as a persistent cryptographic representation of the completed consent authorisation rather than as a transferable asset.



As shown in the figure above, the anchoring of the evaluated consent (token #1) is an ordinary, publicly verifiable transaction: Etherscan reports the status Success, the block number 11,771,243, the transaction fee, and the transfer of one MedicalConsent token to the pseudonymous patient address derived from the patient DID. The transaction can be inspected at [Etherscan: mint transaction](#). This provides an implementation-level demonstration that consent activation can be verified independently of the application.

4. Secure Patient Data Access for Doctor

The doctor enters an authorised Consent ID, after which the system retrieves and displays the associated consent information. The subsystem shows the patient DID, Consent ID, purpose, and current consent state:

State = ACTIVE

This is important because the doctor does not directly access patient data based solely on possession of a Consent ID. The system first evaluates whether the underlying authorisation remains valid, and it does so for every component of the consent object. The decision matrix displayed to the doctor reports the outcome of each check individually.

Enter Authorised Consent ID

f050c29a-3d92-4131-8605-63bcb1ff8d92

Consent Information

Patient

did:pat...

Consent ID

f050c2...

Purpose

Treat...

Requester jurisdiction (VL check)

AU

Decision matrix - ALLOW = VI AND VA AND VP AND VT AND VL

VI	VA	VP	VT	VL	ALLOW
PASS	PASS	PASS	PASS	PASS	PASS

Consent State: ACTIVE

Figure 6. The doctor subsystem with requester jurisdiction AU: all five checks report PASS, ALLOW = PASS, and the consent state is ACTIVE.

As shown in the figure above, the five checks VI (identity), VA (authorisation), VP (purpose), VT (time) and VL (jurisdiction) all report PASS, and consequently ALLOW = PASS. The subsystem then presents the Authorised Data Scope, showing that only the EMR modules specified in the original consent are available. In the demonstrated transaction, the authorised modules are:

Scope = { Observation, Medication, Condition, Procedure }

The doctor must subsequently select the modules to request; the request must be a non-empty subset of the authorised scope. The system displays the requested

modules and performs cryptographic verification before granting access. In this version, the verification also generates and verifies a Groth16 zero-knowledge proof, whose public inputs are a Poseidon commitment and the current time and whose private witnesses are a secret and the expiry time; the proof establishes that the commitment is satisfied and that the current time does not exceed the expiry. Because the prover and the verifier currently run on the same server, the proof demonstrates the integration of the zero-knowledge toolchain rather than an additional privacy guarantee (Section 11). The audit ledger records the proof mode (groth16) with every granted access, and the subsystem then shows the message Cryptographic Verification Successful — Access Granted.

The doctor is therefore authorised only when the requested resources are consistent with the consent object and the consent remains active. The final Authorised Medical Records section presents the actual data modules available to the doctor. Because the subsystem separates the OBSERVATION, MEDICATION, CONDITION and PROCEDURE modules, it provides a direct visual demonstration of fine-grained EMR access control.

Thus, the doctor subsystem corresponds to **Consent-State Validation, Scope Enforcement and Authorised Data Access**.

5. Negative Demonstration: Jurisdiction Violation

The fourth part is the negative demonstration. This is important because a consent framework must be evaluated not only on whether it grants legitimate access, but also on whether it refuses illegitimate access. The same doctor therefore repeated the request for the evaluated consent while declaring the requester jurisdiction as SA, whereas the consent was issued for AU.

The screenshot shows a web interface for a doctor subsystem. At the top, there is a field to 'Enter Authorised Consent ID' with the value 'f050c29a-3d92-4131-8605-63bcb1ff8d92'. Below this is a section titled 'Consent Information' with three cards: 'Patient' with 'did:pat...', 'Consent ID' with 'f050c2...', and 'Purpose' with 'Treat...'. Underneath is a 'Requester jurisdiction (VL check)' dropdown menu set to 'SA'. Below that is a 'Decision matrix - ALLOW = VI AND VA AND VP AND VT AND VL' table. The table has columns for VI, VA, VP, VT, VL, and ALLOW. The rows show 'PASS' for VI, VA, VP, and VT, 'FAIL' for VL, and 'FAIL' for ALLOW. At the bottom, there is a green bar with a green circle and the text 'Consent State: ACTIVE'.

VI	VA	VP	VT	VL	ALLOW
PASS	PASS	PASS	PASS	FAIL	FAIL

Consent State: ACTIVE

Figure 8. The doctor subsystem with requester jurisdiction SA: VL reports FAIL and consequently ALLOW = FAIL, although the consent state remains ACTIVE.

As shown in the figure above, the decision matrix immediately reports VL = FAIL and consequently ALLOW = FAIL, even though the consent itself is still ACTIVE, the guardian signatures remain valid and all other checks pass. This is important because

it demonstrates that the access decision is taken over the complete consent tuple rather than over the consent state alone.

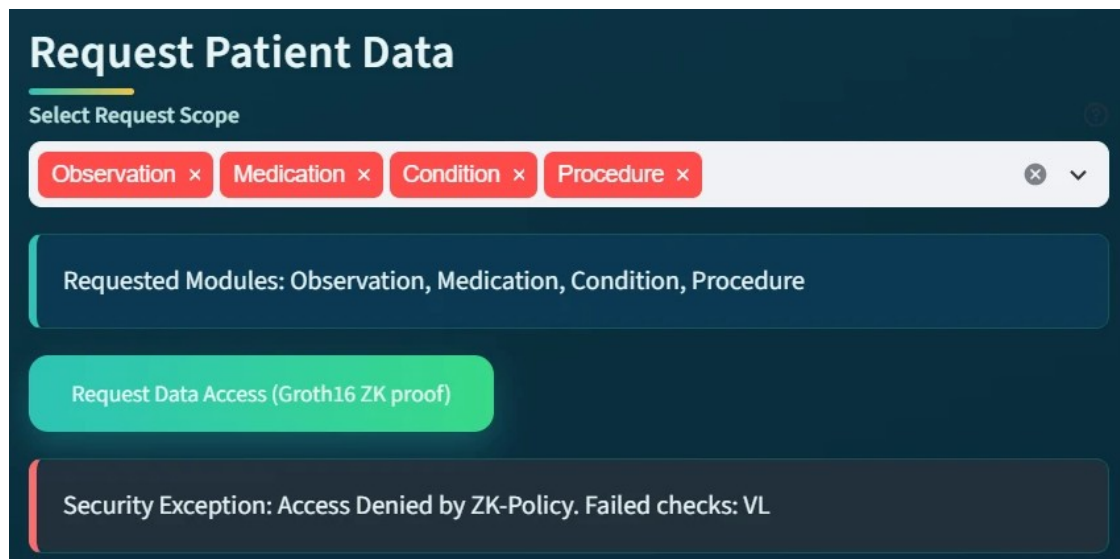


Figure 9. The refused request: 'Security Exception: Access Denied by ZK-Policy. Failed checks: VL'.

As shown in the figure above, when the doctor submits the request, access is refused and the failed check is named explicitly. The corresponding audit-ledger entry records the result DENIED together with the complete decision matrix, so that the refusal is itself auditable. This is important because the jurisdiction check is included in the access decision rather than merely displayed.

This demonstration corresponds to **Jurisdictional Constraint Enforcement and Fail-Closed Access Control**.

6. Consent Revocation

The next module is the Consent Revocation module. This module operationalises the enforcement component of the consent object, which requires that a patient can withdraw consent and that the withdrawal takes effect for every subsequent request. In the patient's My Consents view, each consent is listed with its state, signature count and token. The patient then selects Revoke, after which the system updates the application state and submits a revoke transaction to the contract on Sepolia.

My Consents			
2502632b... → did:hospital:001	REVOKED	0/2 sigs - token: -	REVOKED
aca837a1... → did:hospital:001	REVOKED	0/2 sigs - token: -	REVOKED
cd9a24bd... → did:hospital:001	REVOKED	2/2 sigs - token: SBT-LOCAL- cd9a24bd	REVOKED
011bbb21... → did:hospital:001	REVOKED	2/2 sigs - token: SBT-8	REVOKED
0975c645... → did:hospital:001	PENDING_SIGNED	0/2 sigs - token: -	<button>Revoke</button>
f050c29a... → did:hospital:001	REVOKED	2/2 sigs - token: SBT-1	REVOKED

Figure 10. The patient subsystem after revocation: the evaluated consent f050c29a is REVOKED with token #1 (displayed as SBT-1). The other rows are earlier trial consents.

As shown in the figure above, the revoked consent is displayed as REVOKED together with its on-chain token number. The revocation was confirmed on Sepolia in block 11,771,307, and from that point the contract itself reports the token as no longer valid, so any verifier that reads the chain observes the withdrawal.

Secure Patient Data Access

Enter Authorised Consent ID

Consent Information

Patient did:patient:fe62...	Consent ID f050c29a-3d92-...	Purpose Treatment
---------------------------------------	--	-----------------------------

Requester jurisdiction (VL check)
AU

Decision matrix - ALLOW = VI AND VA AND VP AND VT AND VL

VI	VA	VP	VT	VL	ALLOW
PASS	FAIL	PASS	PASS	PASS	FAIL

Consent is not active. Current state: REVOKED

The Doctor cannot access patient data because the consent is not currently valid.

Figure 11. The doctor subsystem after revocation: VA reports FAIL and the consent is reported as 'not active. Current state: REVOKED'.

As shown in the figure above, when the doctor subsequently enters the same Consent ID, the authorisation check VA fails and the subsystem reports that the consent is not active. Consequently, revocation is not merely a change of display: it removes the authorisation on which every later access decision depends.

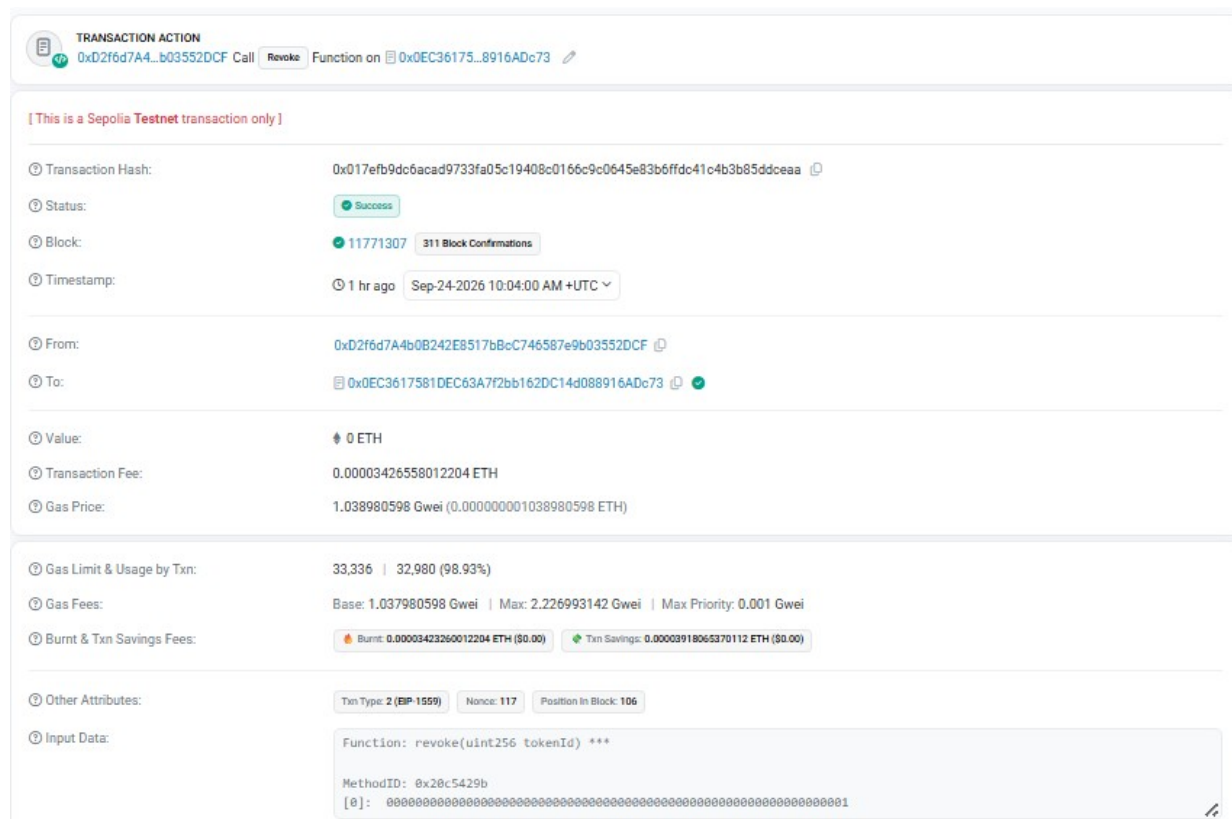


Figure 12. The revocation transaction on Sepolia Etherscan: status Success, block 11,771,307, decoded input 'Function: revoke(uint256 tokenId)' with tokenId = 1.

As shown in the figure above, the revocation is again a publicly verifiable transaction, whose decoded input names the contract function revoke and the token identifier. The transaction can be inspected at [Etherscan: revoke transaction](#). The revocation subsystem therefore corresponds to **Enforcement State Management and Consent Withdrawal**.

7. Blockchain Audit & Consent Lifecycle for Auditor

The fifth major module is the Blockchain Audit & Consent Lifecycle module. This subsystem provides an independent view of the complete consent process and corresponds to the auditability and accountability component of the HALAH framework.

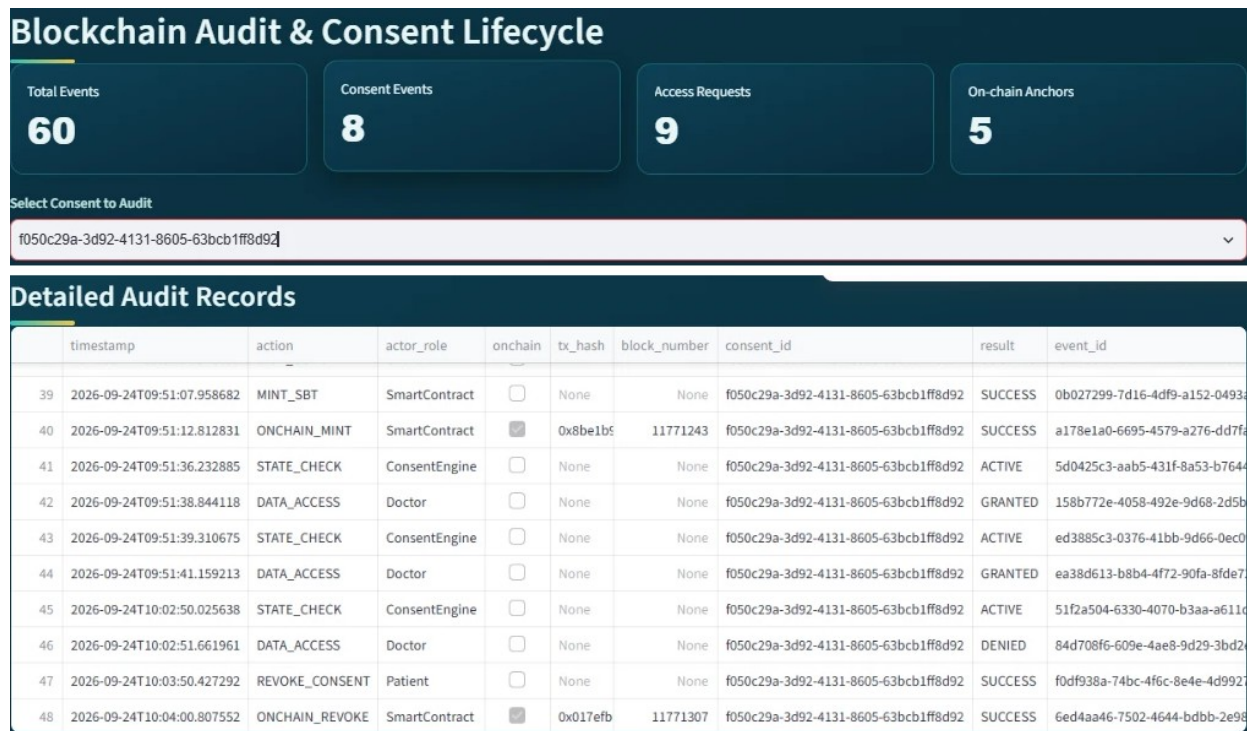


Figure 13. The auditor subsystem with the evaluated consent selected: aggregate indicators and the Detailed Audit Records, in which the on-chain events carry their transaction hashes and block numbers.

As shown in the figure above, the system first provides aggregate indicators including:

Total Events = 60 Consent Events = 8 Access Requests = 9 On-chain Anchors = 5

These indicators give the auditor an immediate overview of the recorded system activity and of how many events are anchored on the public chain. The five anchors comprise the mint and revocation of the evaluated consent (token #1), the mint of the photographed lifecycle (token #2), and the mint and revocation of an earlier trial consent on the previous contract; the nine access requests include the granted, refused and reproduced requests of both lifecycles. More importantly, the Consent Lifecycle section reconstructs the sequence of events associated with a particular consent. The reconstructed records include CREATE_CONSENT performed by the patient, followed by SIGN_CONSENT performed by each guardian, followed by MINT_SBT and ONCHAIN_MINT performed by the smart contract; in the figure the detailed table is scrolled to the on-chain events, whose rows carry the transaction hashes. The subsequent system events STATE_CHECK demonstrate that the Consent Engine continuously evaluates the state of the authorisation. The doctor then generates DATA_ACCESS with the result GRANTED, later DATA_ACCESS with the result DENIED when the jurisdiction check fails, and finally the patient generates REVOKE_CONSENT followed by ONCHAIN_REVOKE performed by the smart contract.

This produces an explicit chain of evidence as

Patient → Guardian_A → Guardian_B → SmartContract(mint) → ConsentEngine → Doctor(GRANTED)

→ Doctor(DENIED) → Patient(revoke) → SmartContract(revoke)

The audit subsystem therefore allows an auditor to reconstruct not only who accessed the data, but also why that access was permitted or refused, because each access event can be associated with the preceding consent creation, guardian approvals, on-chain activation, state checks and the decision matrix that was in force at that moment.

This corresponds to the proposed requirement for **traceability, accountability and non-repudiable consent evidence**.

8. Detailed Audit Records

The Detailed Audit Records component provides a lower-level view of the events represented in the lifecycle subsystem.

Where the lifecycle view is primarily designed to communicate the chronological sequence of the authorisation process, the detailed audit view provides the underlying event-level evidence. Each event can be associated with attributes such as actor, role, action, result, timestamp, consent context and, for on-chain events, the transaction hash and block number. As shown in the auditor figure above, the ONCHAIN_MINT and ONCHAIN_REVOKE rows carry the hashes of the two Sepolia transactions presented in Sections 3 and 6, so the ledger and the public chain describe the same events.

This separation is useful because different stakeholders require different levels of information. A clinician may only need to know that a consent is active and valid, whereas an auditor may need to establish precisely when an approval occurred, which identity performed it, and what authorisation decision followed.

The subsystem therefore implements the distinction between operational access information and forensic/audit information.

9. Complete Blockchain Audit Ledger and Independent Verification

The Complete Blockchain Audit Ledger provides the most comprehensive representation of the recorded consent history. This component demonstrates that the audit mechanism is not limited to successful data-access events. Instead, the system records the broader lifecycle of the consent, including creation, signatures, state validation, smart-contract actions, refused requests and revocations. Consequently, the ledger provides the evidentiary foundation for evaluating several properties of the proposed framework:

Accountability + Traceability + Integrity + Non-repudiation

A ledger kept by the application could in principle contain fabricated entries. Consequently, the implementation provides an independent verification tool. The tool

re-reads the ledger and, for every event that carries a transaction hash, fetches the corresponding receipt directly from Sepolia and checks that the transaction exists, succeeded, and was included in the recorded block; for every anchored consent it then reads the token back from the contract and compares the on-chain revocation state with the local state.

```
@yasir12345678900 →/workspaces/HealthRWA-Sepolia-
Personal-try (claude/pensive-tesla-u9uk4j) $ pytho
n -m tools.verify_poc
    local ledger state revoked=false (matches chain
: True)

RESULT: ALL CHECKS PASSED
@yasir12345678900 →/workspaces/HealthRWA-Sepolia-
○ Personal-try (claude/pensive-tesla-u9uk4j) $ █
```

Figure 14. The final verdict of the independent verification executed against Sepolia: the visible line confirms the unrevoked second token against the chain, and the tool reports *RESULT: ALL CHECKS PASSED*.

As shown in the figure above, the verification reports ALL CHECKS PASSED: every recorded transaction was found on Sepolia in its recorded block, and for both tokens the revocation state stored by the application matches the state stored by the contract (token #1 revoked, token #2 not revoked). For example, the demonstrated sequence shows that the doctor receives a GRANTED result only while the consent is in the ACTIVE state, and a DENIED or 'not active' result otherwise. This provides an implementation-level demonstration that access decisions are linked to the consent lifecycle rather than being independent application events, and that the recorded lifecycle is consistent with the public chain.

10. Correspondence Between Subsystems and the HALAH Framework

The subsystems collectively implement the principal stages of the proposed framework:

HALAH Framework Component	Implementation Subsystem	Main Function
Consent Object Construction	Create Smart Consent Contract	Defines patient, authorised doctor, data scope, guardians, jurisdiction and temporal validity
Identity & Authorisation	Guardian Cryptographic Approval	Verifies guardian identities and EIP-712 signatures
Multi-party Consent	Guardian Cryptographic Approval	Implements the 2-of-2 approval condition

HALAH Framework Component	Implementation Subsystem	Main Function
Consent Activation	Guardian Cryptographic Approval	Mints the SBT on Sepolia and marks the consent as fully activated
Consent-State Management	Secure Patient Data Access	Checks whether the consent is ACTIVE
Scope Enforcement	Secure Patient Data Access	Restricts access to authorised EMR modules
Cryptographic Access Control	Secure Patient Data Access	Verifies the request, including a Groth16 proof, before granting data
Jurisdictional Constraint (L)	Secure Patient Data Access	Refuses requests whose declared jurisdiction differs from the consent
Healthcare Data Access	Secure Patient Data Access	Provides the authorised Observation, Medication, Condition and Procedure records
Enforcement / Revocation (E)	My Consents (Patient)	Revokes the consent locally and on-chain
Lifecycle Traceability	Blockchain Audit & Consent Lifecycle	Reconstructs the chronological consent process
Accountability	Detailed Audit Records	Provides event-level evidence with transaction hashes
Ledger Integrity	Complete Ledger + independent verification	Confirms the recorded history against Sepolia

This mapping demonstrates that the implementation is not merely a collection of independent user-subsystem modules. Rather, the modules correspond to successive stages of the proposed authorisation protocol.

11. Overall System Demonstration

Taken together, the subsystems demonstrate an end-to-end implementation of the proposed HALAH authorisation workflow on a public blockchain. The patient first establishes the consent conditions through the Smart Consent Contract subsystem. The designated guardians then independently authenticate and provide the required cryptographic approvals. Once the multi-signature condition is satisfied, the consent is activated and represented by an SBT minted on Sepolia. The authorised doctor subsequently submits an access request, which is evaluated against the patient's identity, consent state, authorised data scope, temporal validity and jurisdiction. Only when these conditions are satisfied does the system provide access to the specified EMR modules; an out-of-jurisdiction request is refused with the failed check named. The patient finally revokes the consent, the revocation is recorded on-chain, and the

complete sequence is exposed through the auditor subsystem and re-verified independently against the chain.

C → MS → SBT_chain → StateCheck → AccessDecision → EMR → Revoke_chain → Audit → Verify

where C denotes consent creation, MS multi-signature approval, SBT_chain on-chain consent activation, StateCheck lifecycle validation, AccessDecision authorisation enforcement over the full consent tuple, EMR authorised medical-record access, Revoke_chain on-chain revocation, Audit immutable lifecycle recording, and Verify independent verification against Sepolia.

The demonstrated prototype makes several simplifications that are acceptable for a research demonstration but would require strengthening before deployment. The guardian signing keys are derived deterministically from the guardian DIDs so that the prototype can sign without wallets; in production each guardian would sign the same typed data with a wallet-held key registered against their DID. The doctor's identity and jurisdiction are declared in the interface rather than authenticated, and the Groth16 proof is generated and verified by the same server, so it currently demonstrates the integration of the zero-knowledge toolchain rather than an additional privacy guarantee. These aspects define the immediate future work of the project.

From a research-implementation perspective, the most significant feature of the presented subsystems is therefore the continuity between consent specification, multi-party authorisation, public on-chain activation and revocation, access enforcement and independently verifiable audit evidence. The user subsystems provide a visible representation of this continuity, allowing the proposed HALAH model to be demonstrated as an operational authorisation workflow on a public network rather than only as a conceptual framework.

Appendix: Verification Links

Contract:

<https://sepolia.etherscan.io/address/0x0EC3617581DEC63A7f2bb162DC14d088916ADc73>

Consent activation (mint of token #1):

<https://sepolia.etherscan.io/tx/0x8be1b99e9b7054d427bdee0a0b5cc56ae99030a399f4b11d3629ed5d98daf13d>

Consent revocation (token #1):

<https://sepolia.etherscan.io/tx/0x017efb9dc6acad9733fa05c19408c0166c9c0645e83b6ffdc41c4b3b85ddceaa>

Full audit-ledger export: evidence/sepolia_audit_ledger_2026-09-24.csv;
deployment and run record: evidence/sepolia_deploy_v3_2026-09-24.txt (repository branch claude/pensive-tesla-u9uk4j).